Early Feedback Survey

https://go.blueja.io/acJt2Tr5FkOyoJetPl5y7w

- Completely anonymous
- 4 questions to be answered (5-10 min)
- All feedback are welcomed

Machine Learning Algorithms and Applications

UPASS

UPASS Sessions (starting next week)
with **Kelly**

on

- **Tuesdays 3-4pm CB.**
- **Fridays 4-5pm CB.05D.02.020**

www.uts.edu.au/upass-sessions

You can mention what topics you want to cover next sessions by filling this form:

https://www.menti.com/al22hkg2o2sa?source=qr-page

UTS
UNIVERSITY OF TECHNOLOGY SYDNEY

Machine Learning Algorithms and Applications

Lecture 4

(Block 2)

Plan for Today

Binary Classification

Support Vector Machine

Accuracy Metric

Feature Scaling

Lab

4

Binary Classification

What is a Binary Classification Task

Supervised learning
Requires a target variable containing the true values.

Binary Variable
The target variable can only take 2 possible values: 0 or 1 (false or true, no or yes)

Objective
Find a model that will approximate the correlation between the target and features

Binary Classification Application

(Retail) Predict promotional email opening: yes or no

(Telco) Predict customer churn : yes or no

(Finance) Detecting fraudulent transaction: yes or no

(Medical) Predict if a patient will be sick: yes or no

...

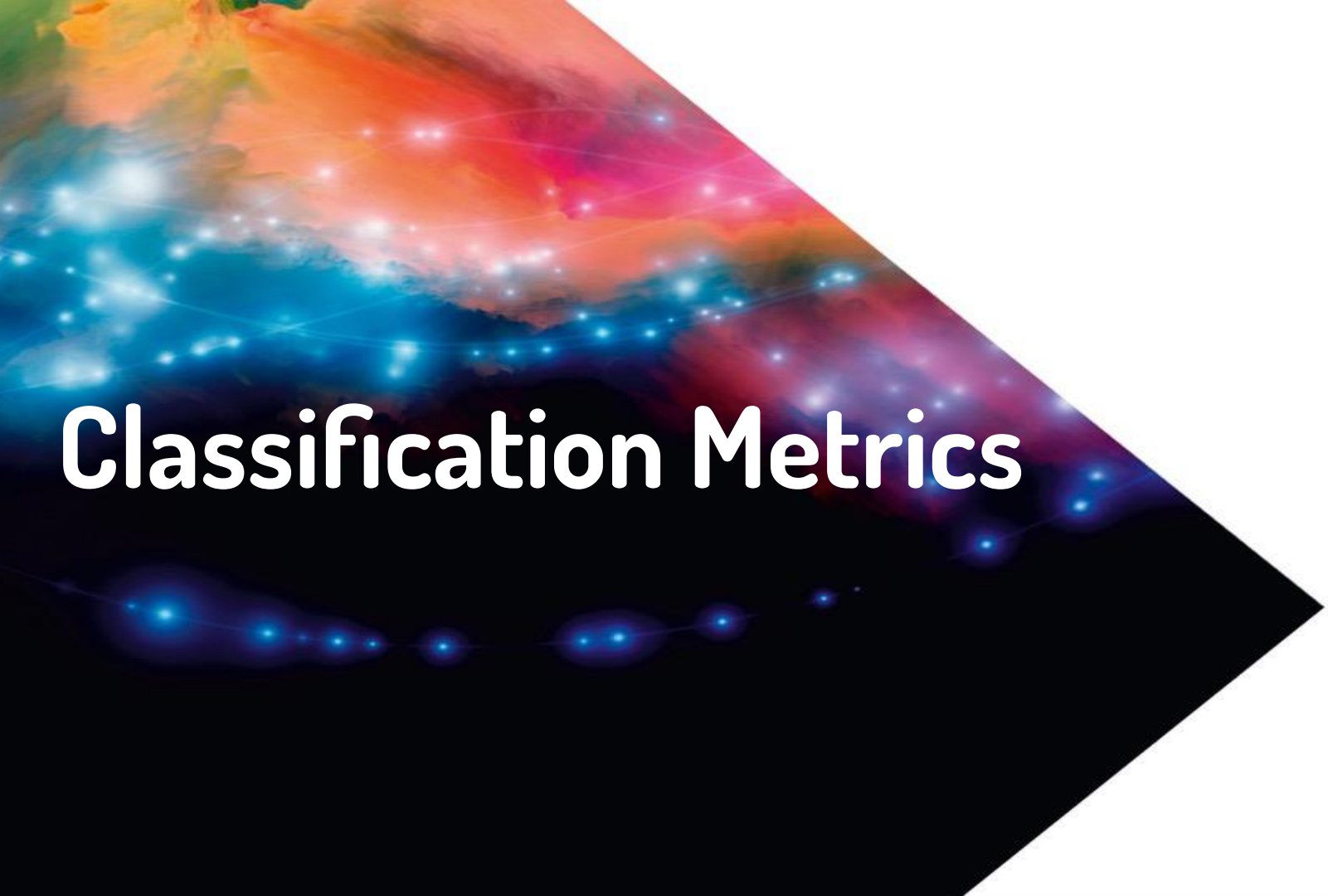
Binary Classification

Predicting Customer Churn

Target

Features

Cancel	Months since first subscription	Monthly average spent	Average number of phone calls made last month	Average number of phone calls made last quarter	Additional options
Yes	13	$70	56	63	Yes
No	2	$35	35	34	Yes
No	6	$40	46	50	Yes
Yes	16	$110	53	75	No



Classification Metrics

Accuracy

Formula:

$$accuracy = \frac{number\ of\ correct\ predictions}{total\ number\ of\ observations}$$

Binary Classification:

⇨ Number of correct predictions = all true positive and all true negative

Accuracy = 0 (Worst score)

Accuracy = 1 (Perfect score)

Accuracy

xi	Y_i	$\hat{Y}_i$
1	0	1
2	0	0
3	1	1
4	1	1
5	1	0

Correct predictions
3

Observations
5

Accuracy
3/5 = 0.60

sklearn:

```
from sklearn.metrics import accuracy_score
y_pred = [1, 0, 1, 1]
y_true = [0, 0, 1, 1]
accuracy_score(y_true, y_pred)
```

```
0.75
```

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html?highlight=accuracy#sklearn.metrics.accuracy_score

Null Accuracy

Definition: Accuracy of the model always predicting the mode (value that appears the most in dataset

xi	Y_i	$\hat{Y}_i$
1	0	1
2	0	1
3	1	1
4	1	1
5	1	1

Correct predictions
3

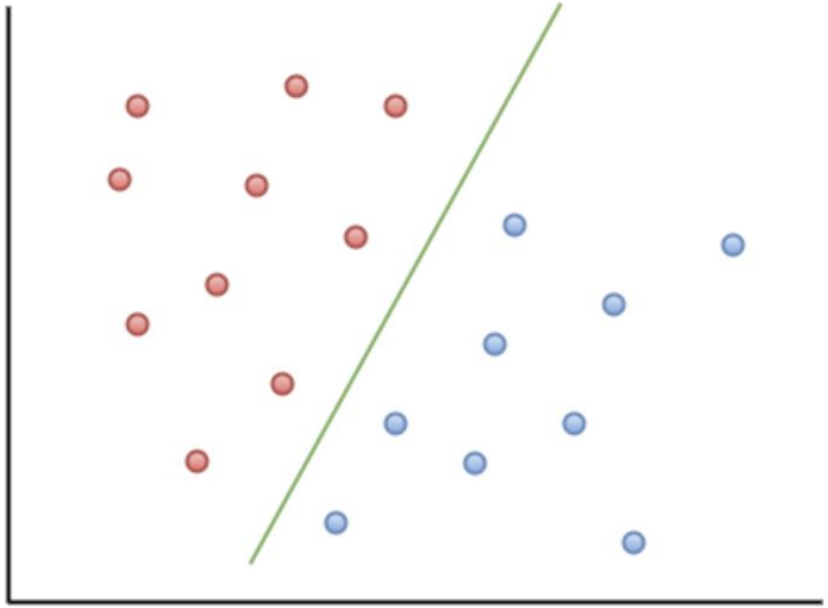
Observations
5

Accuracy
3/5 = 0.60

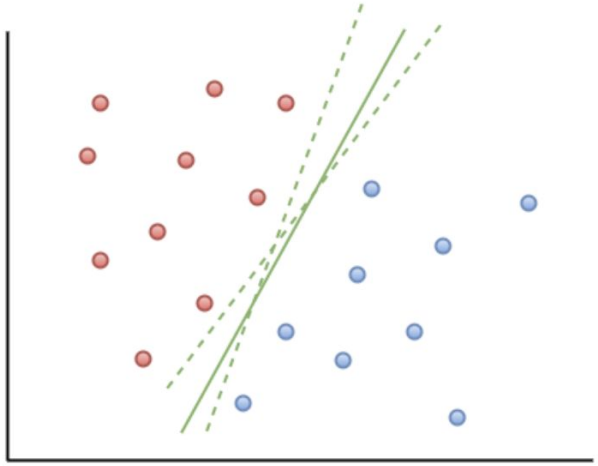
Used as a baseline to compare model performance

Support Vector Machine

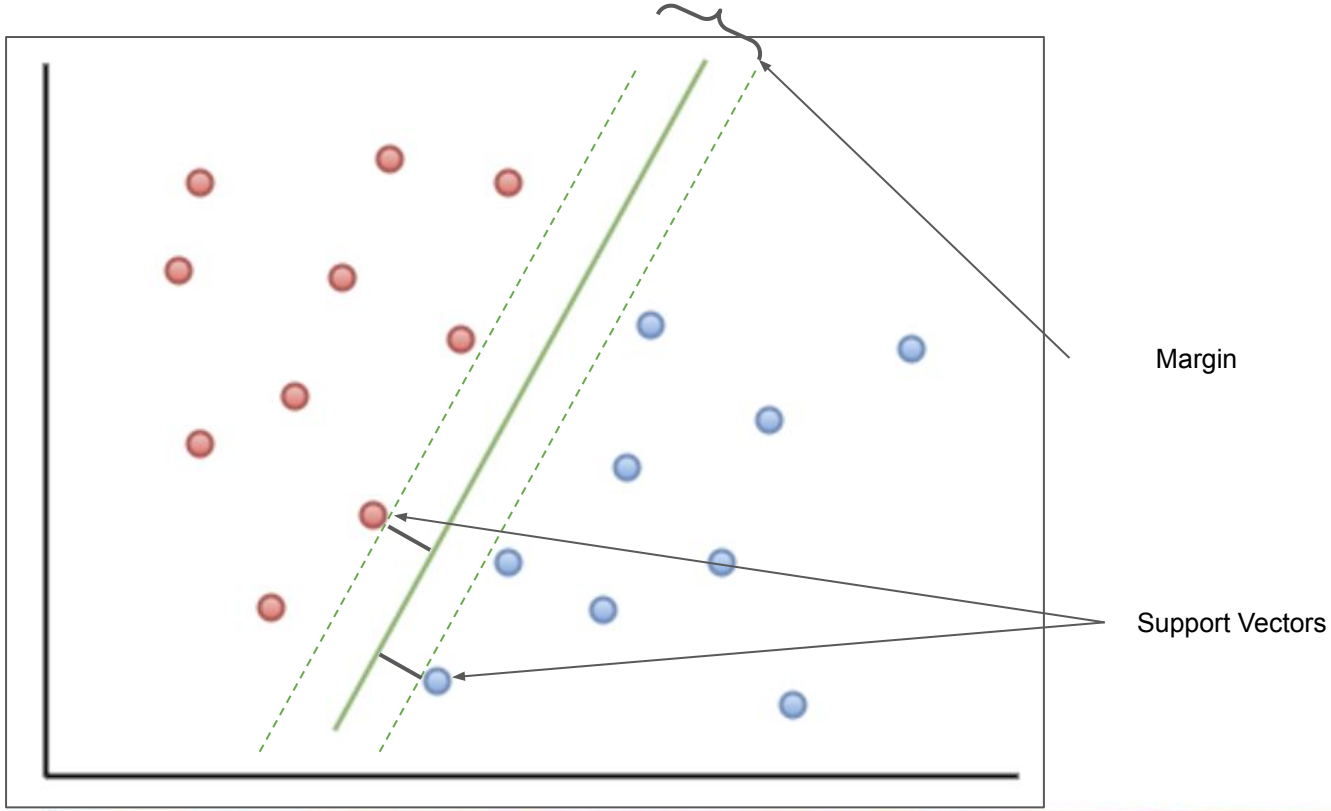
SVM for Classification



Objective:
Find the best boundary that will separate classes

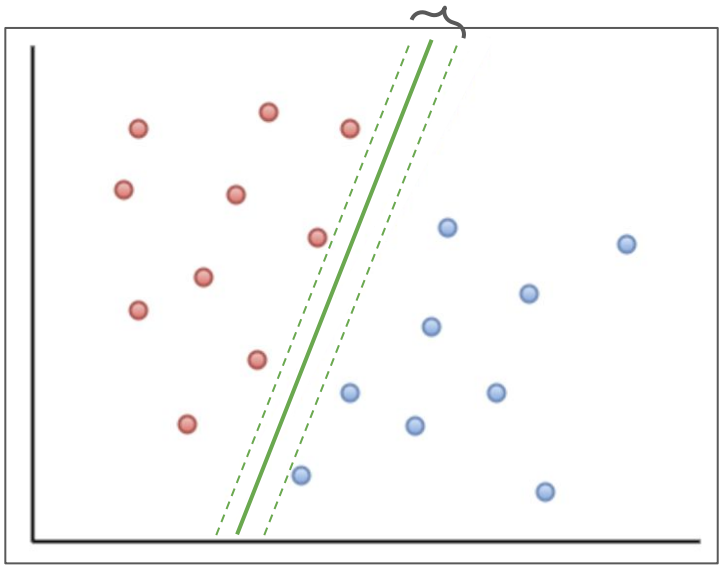


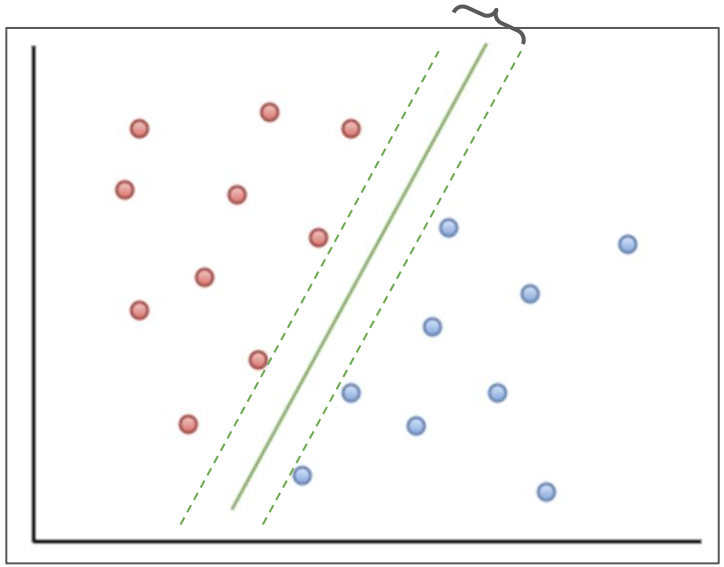
Finding Boundary



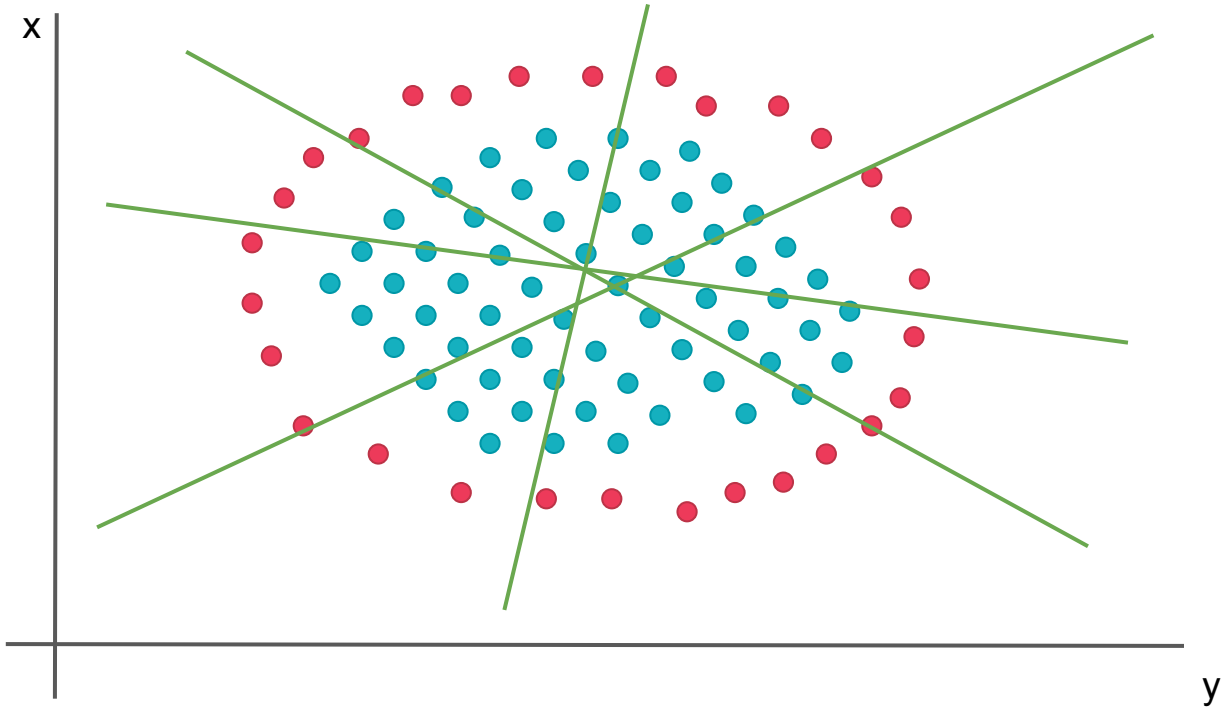
Objective:
Find the best boundary that will separate classes (biggest margin between support vectors)

Best Boundary

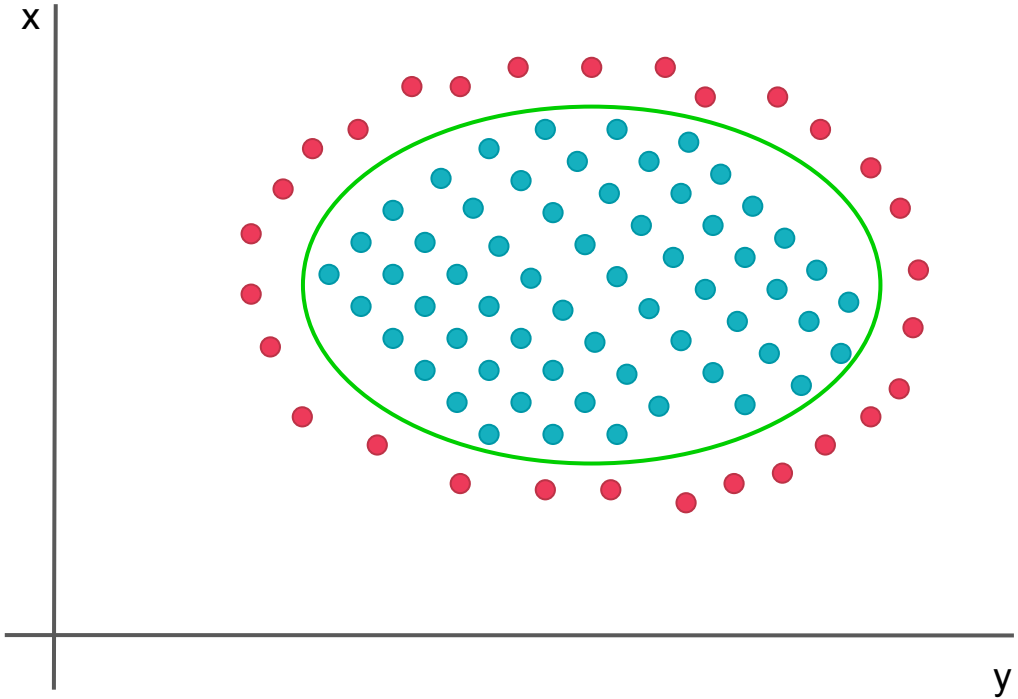




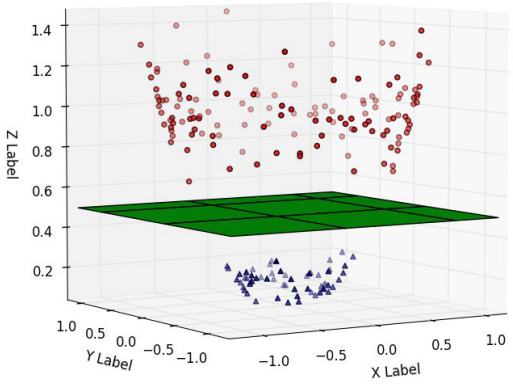
What if not linearly separable



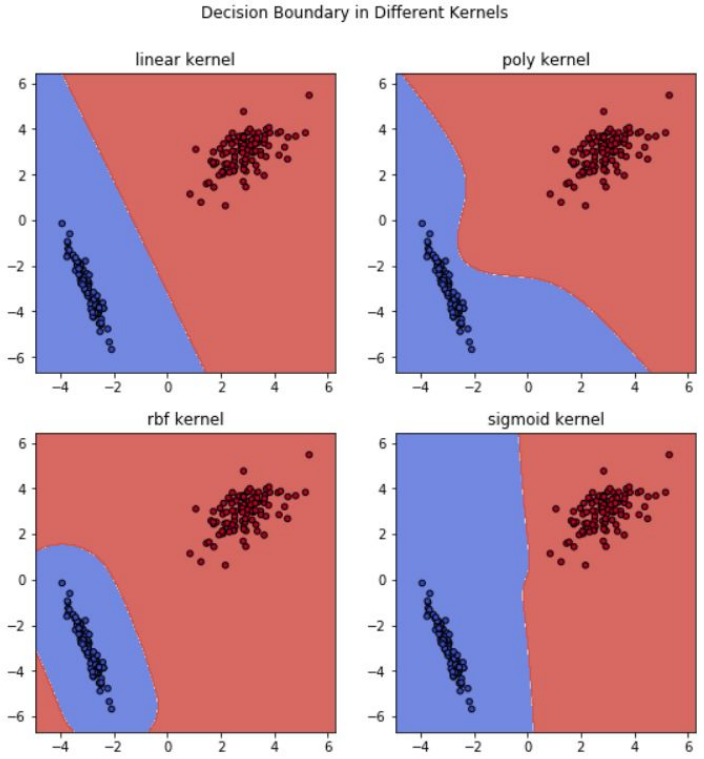
Kernel Trick



Solution:
Add a new dimension (by applying a transformation) and find the best separating hyperplane



Kernel Hyperparameter

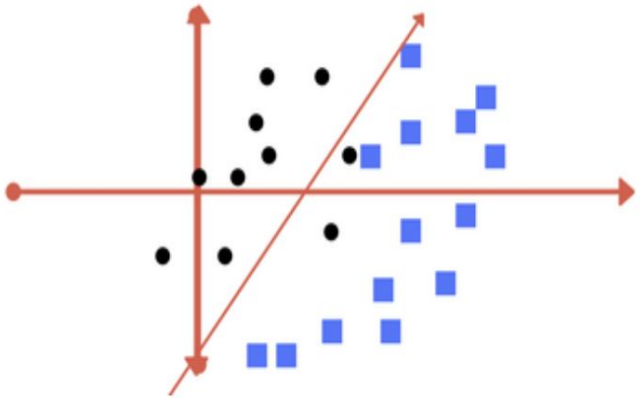


Type of transformation applied for the kernel trick (adding a new dimension):

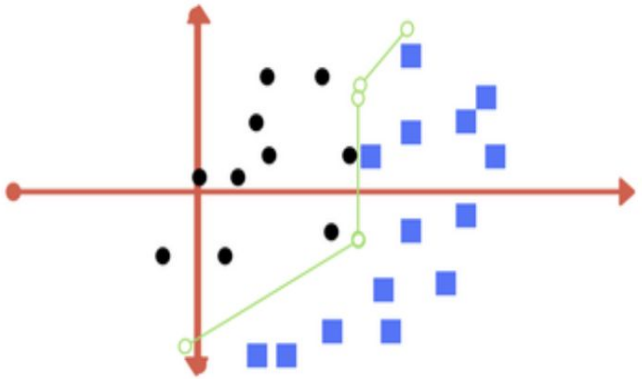
- Linear
- Polynomial
- Radial basis function
- Sigmoid

Regularization Hyperparameter

Also called C. Penalty applied for misclassification.



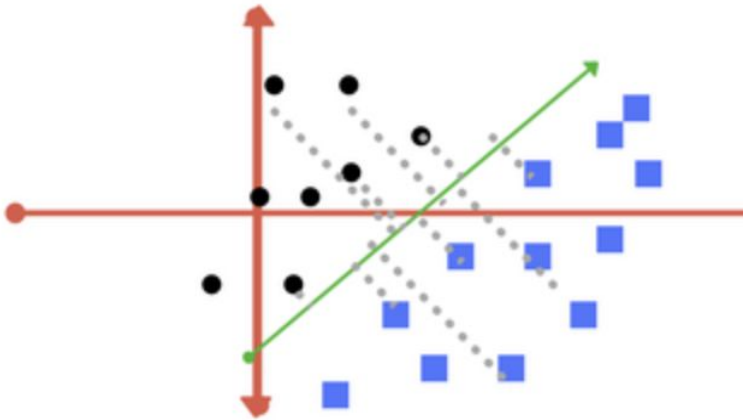
Low value of C => higher margin



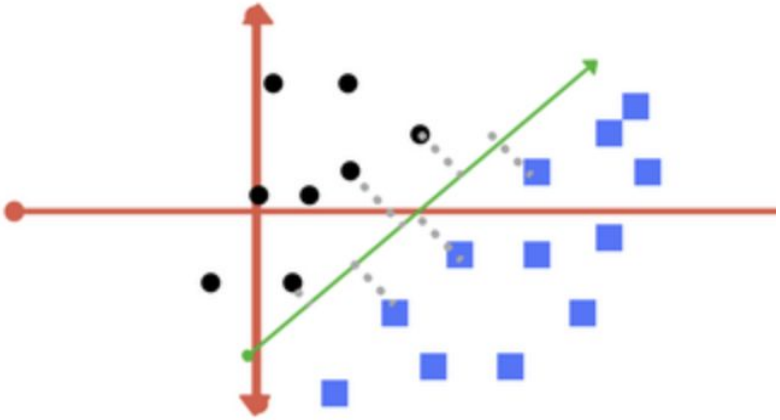
High value of C => smaller margin

Gamma Hyperparameter

How much support vectors are taken into consideration to determine the best boundary



Low value of gamma => further points are used



High value of gamma => only closer points

Scikit-learn SVC

Generic API for sklearn models:

Import class

```
from sklearn.svm import SVC
```

Instantiate a model with hyperparameter

```
svc = SVC(C=1000.0, kernel='rbf', gamma=0.005)
```

Fit a model

```
svc.fit(X, y)
```

Predict with a model

```
svc.predict(new_X)
```

Sklearn Parameters

https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html?highlight=svc#sklearn.svm.SVC

Parameters:

C : *float, default=1.0*
Regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive. The penalty is a squared l2 penalty.

kernel : *{'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'}, default='rbf'*
Specifies the kernel type to be used in the algorithm. It must be one of 'linear', 'poly', 'rbf', 'sigmoid', 'precomputed' or a callable. If none is given, 'rbf' will be used. If a callable is given it is used to pre-compute the kernel matrix from data matrices; that matrix should be an array of shape `(n_samples, n_samples)`.

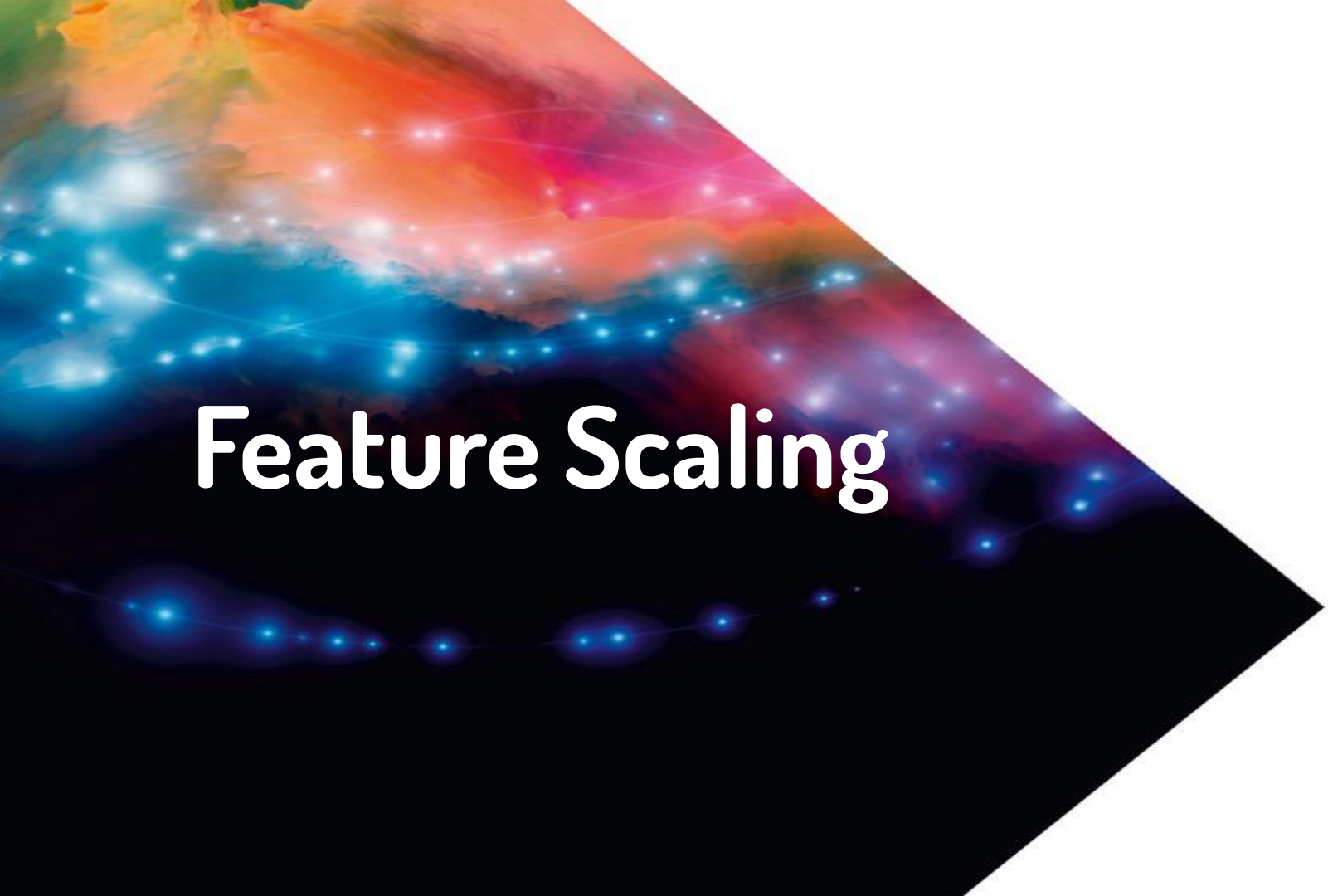
degree : *int, default=3*
Degree of the polynomial kernel function ('poly'). Ignored by all other kernels.

gamma : *{'scale', 'auto'} or float, default='scale'*
Kernel coefficient for 'rbf', 'poly' and 'sigmoid'.

- if `gamma='scale'` (default) is passed then it uses 1 / (n_features * X.var()) as value of gamma,
- if 'auto', uses 1 / n_features.

Changed in version 0.22: The default value of `gamma` changed from 'auto' to 'scale'.

coef0 : *float, default=0.0*
Independent term in kernel function. It is only significant in 'poly' and 'sigmoid'.



Feature Scaling

Why Scaling

Example: Euclidean Distance

$$A = [110000,\ 1.0]$$

$$B = [100000,\ 0.5]$$

$$d(A,B) = \sqrt{(110000 - 100000)^2 + (1 - 0.5)^2}$$

$$d(A,B) = \sqrt{(10000)^2 + (0.5)^2}$$

$$d(A,B) \simeq \sqrt{(10000)^2} \simeq 10000$$

The difference of scale between variables will lead to some of them being neglected

Scaling all the features to similar ranges will allow models to take into account each of them for predictions

Min-Max Scaling

Formula:

$$X_{sc} = \frac{X - X_{min}}{X_{max} - X_{min}}.$$

Output: between 0 and 1

Example:

$$X = [-1, 0, 1, 2]$$

$$X_{sc} = \frac{-1-(-1)}{2-(-1)} = \frac{0}{3} = 0$$

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html?highlight=minmax#sklearn.preprocessing.MinMaxScaler

```
from sklearn.preprocessing import MinMaxScaler
```

```
min_max_scaler = MinMaxScaler()
```

```
min_max_scaler.fit(X)
```

```
MinMaxScaler(copy=True, feature_range=(0, 1))
```

```
min_max_scaler.transform(X)
```

```
array([[0.        ],
       [0.33333333],
       [0.66666667],
       [1.        ]])
```

Max-Abs Scaling

Formula:

$$X_{sc} = \frac{X}{|X_{max}|}$$

Output: between -1 and 1

Example:

$$X = [-1, 0, 1, 2]$$

$$X_{sc} = \frac{-1}{|2|} = -0.5$$

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MaxAbsScaler.html?highlight=max%20abs#sklearn.preprocessing.MaxAbsScaler

```
from sklearn.preprocessing import MaxAbsScaler
```

```
max_abs_scaler = MaxAbsScaler()
```

```
max_abs_scaler.fit(X)
```

```
MaxAbsScaler(copy=True)
```

```
max_abs_scaler.transform(X)
```

```
array([[-0.5],
       [ 0. ],
       [ 0.5],
       [ 1. ]])
```

Standardisation Scaling

Formula:

$$X_{sc} = \frac{X - \bar{x}}{\sigma}$$

Output: mean of 0 and standard deviation of 1

Example:

$$X = [-1, 0, 1, 2]$$

$$X_{sc} = \frac{-1 - 0.5}{1.118} = -1.34$$

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html?highlight=standardscaler#sklearn.preprocessing.StandardScaler

```
from sklearn.preprocessing import StandardScaler
```

```
std_scaler = StandardScaler()
```

```
std_scaler.fit(X)
```

```
StandardScaler(copy=True, with_mean=True, with_std=True)
```

```
std_scaler.transform(X)
```

```
array([[-1.34164079],
       [-0.4472136 ],
       [ 0.4472136 ],
       [ 1.34164079]])
```

Time for Lab

29